May 2025 iOS Interview Handbook
Advanced Swift Interview Topics for iOS Developers

How would you securely store sensitive user data on-device?

Description:

Sensitive user data like auth tokens, passwords, or biometric preferences **must be stored encrypted**. Apple provides the **Keychain** for this. If you need to store files or larger data, use **File Protection APIs** with `.completeFileProtection`.

Sample (Using KeychainAccess Library):

```
import KeychainAccess

let keychain = Keychain(service: "com.yourapp.bundleid")

do {
    try keychain.set("my-secret-token", key: "userToken")
    let token = try keychain.get("userToken")
    print("Retrieved token: \(token ?? "nil")")
} catch {
    print("Keychain error: \(error)")
}
```

Use KeychainAccess via Swift Package Manager or CocoaPods.

What is Keychain, and how is it used in iOS?

Description:

The **Keychain** is Apple's encrypted database used to store credentials, tokens, certificates, and other secrets securely. It's persistent and sandboxed to your app unless explicitly shared.

Sample (Using native API):

```
import Security
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
func saveToKeychain(value: String, forKey key: String) {
    let data = Data(value.utf8)
    let query: [String: Any] = [
        kSecClass as String: kSecClassGenericPassword,
        kSecAttrAccount as String: key,
        kSecValueData as String: data
    ]
    SecItemAdd(query as CFDictionary, nil)
}

func getFromKeychain(forKey key: String) -> String? {
    let query: [String: Any] = [
        kSecClass as String: kSecClassGenericPassword,
        kSecAttrAccount as String: key,
        kSecReturnData as String: true,
        kSecMatchLimit as String: kSecMatchLimitOne
    ]
    var item: CFTypeRef?
    let status = SecItemCopyMatching(query as CFDictionary, &item)

    guard status == errSecSuccess, let data = item as? Data else {
return nil }
    return String(data: data, encoding: .utf8)
}
```

What's the difference between symmetric and asymmetric encryption?

Description:

- **Symmetric Encryption (e.g., AES)**:
    - Same key used for encryption and decryption.
    - Fast, good for encrypting large data.
- **Asymmetric Encryption (e.g., RSA, ECC)**:
    - Public key encrypts, private key decrypts.
    - Secure for communication, slower than symmetric.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Feature	Symmetric (AES)	Asymmetric (RSA)
Keys	One (shared)	Two (public/private)
Speed	Fast	Slower
Use Case	Bulk encryption	Key exchange, digital signatures

How would you implement encryption for data sent to a server?

Description:

Always use **HTTPS (TLS)**. For an additional encryption layer:

- Encrypt payload with **AES**
- Encrypt AES key with **server's public RSA key**
- Server uses private key to decrypt AES key and then decrypts payload

Sample (AES using CryptoKit):

```
import CryptoKit

func encryptWithAES(message: String, key: SymmetricKey) -> Data {
    let data = Data(message.utf8)
    let sealedBox = try! AES.GCM.seal(data, using: key)
    return sealedBox.combined!
}

func decryptWithAES(ciphertext: Data, key: SymmetricKey) -> String? {
    let sealedBox = try! AES.GCM.SealedBox(combined: ciphertext)
    let decryptedData = try! AES.GCM.open(sealedBox, using: key)
    return String(data: decryptedData, encoding: .utf8)
}

// Usage
let key = SymmetricKey(size: .bits256)
let encrypted = encryptWithAES(message: "Hello, server!", key: key)
let decrypted = decryptWithAES(ciphertext: encrypted, key: key)
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
print(decrypted ?? "decryption failed")
```

What iOS frameworks do you use for cryptographic operations?

Description:

- **CryptoKit** (modern Swift API for cryptography, available from iOS 13+)
- **Security framework** (Keychain, certificate management, RSA/EC)
- **CommonCrypto** (older C-based APIs for lower-level needs)

CryptoKit Features:

- AES-GCM encryption
- SHA hashing
- Curve25519 key agreement
- HMAC, signing

Example (Hashing with SHA256 using CryptoKit):

```
import CryptoKit

let input = "password123"
let inputData = Data(input.utf8)
let hash = SHA256.hash(data: inputData)

let hashString = hash.compactMap { String(format: "%02x", $0)
}.joined()
print("SHA256 Hash: \(hashString)")
```

---

sample project combining Keychain, AES encryption, and a simple secure API call?

1. **Store sensitive data in the Keychain**
2. **Encrypt data using AES (CryptoKit)**
3. **Send encrypted data to a mock API**
4. **(Optional)** Show how the server would decrypt it (conceptually)

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Project Name: SecureDataDemo

Project Structure:

```
SecureDataDemo/
├── SecureStorage.swift          ← Keychain functions
├── EncryptionManager.swift      ← AES encryption/decryption
├── ViewController.swift         ← UI + integration
└── (Optionally) ServerMock.swift  ← Simulated server decryption
```

SecureStorage.swift (Keychain helper)

```
import Foundation
import Security

class SecureStorage {
    static func save(value: String, forKey key: String) {
        let data = Data(value.utf8)
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key,
            kSecValueData as String: data
        ]
        SecItemDelete(query as CFDictionary) // Remove old item if
exists
        SecItemAdd(query as CFDictionary, nil)
    }

    static func load(forKey key: String) -> String? {
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key,
            kSecReturnData as String: true,
            kSecMatchLimit as String: kSecMatchLimitOne
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
        ]

        var item: CFTypeRef?
        let status = SecItemCopyMatching(query as CFDictionary, &item)

        guard status == errSecSuccess, let data = item as? Data else {
return nil }
        return String(data: data, encoding: .utf8)
    }
}
```

`EncryptionManager.swift` (CryptoKit AES)

```
import Foundation
import CryptoKit

class EncryptionManager {
    static let shared = EncryptionManager()
    private let symmetricKey = SymmetricKey(size: .bits256)

    func encrypt(message: String) -> Data? {
        let data = Data(message.utf8)
        guard let sealedBox = try? AES.GCM.seal(data, using:
symmetricKey) else { return nil }
        return sealedBox.combined
    }

    func decrypt(ciphertext: Data) -> String? {
        guard let sealedBox = try? AES.GCM.SealedBox(combined:
ciphertext),
              let decryptedData = try? AES.GCM.open(sealedBox, using:
symmetricKey)
        else {
            return nil
        }
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
        return String(data: decryptedData, encoding: .utf8)
    }
}
```

`ViewController.swift` (Demo use)

```
import UIKit

class ViewController: UIViewController {

    override func viewDidLoad() {
        super.viewDidLoad()
        simulateSecureDataFlow()
    }

    func simulateSecureDataFlow() {
        let token = "super_secure_token_9876"
        SecureStorage.save(value: token, forKey: "authToken")

        if let storedToken = SecureStorage.load(forKey: "authToken") {
            print(" Token from Keychain: \(storedToken)")

            // Encrypt token before sending to API
            if let encrypted =
EncryptionManager.shared.encrypt(message: storedToken) {
                print("🔐 Encrypted Data:
\(encrypted.base64EncodedString())")

                // Mock "sending" to API...
                let serverReceived = encrypted

                // Decrypt on server side
                if let decrypted =
EncryptionManager.shared.decrypt(ciphertext: serverReceived) {
                    print(" Decrypted on server: \(decrypted)")
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
                }
            }
        }
    }
}
```

(Optional) `ServerMock.swift`

This is purely for demo. In real world, server uses its **own symmetric or private key**.

Requirements:

iOS 13+

Swift 5+

Add `CryptoKit` (built-in)

Add KeychainAccess if you want a cleaner API (optional)

What is a Closure in Swift?

Definition:

A **closure** is a self-contained block of functionality that can be passed around and used in your code. It's similar to a function but can **capture and store references to variables and constants** from the context in which it is defined.

Closures can:

- Be stored in variables
- Be passed as parameters
- Return values
- Capture external values

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Syntax Example:

```
let greet = { (name: String) -> String in
    return "Hello, \(name)!"
}

print(greet("Ruchit")) // Output: Hello, Ruchit!
```

Real-World Example:

Sorting an array of names using closures:

```
let names = ["Zara", "Ruchit", "Amit", "Bhavik"]
let sorted = names.sorted(by: { $0 < $1 })
print(sorted)
```

What is a Trailing Closure?

Definition:

A **trailing closure** is a closure written **outside the parentheses** of a function call, especially useful when the closure is the **last argument**.

Example (with and without trailing syntax):

Without trailing closure:

```
UIView.animate(withDuration: 0.3, animations: {
    view.alpha = 0
})
```

With trailing closure:

```
UIView.animate(withDuration: 0.3) {
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
    view.alpha = 0
}
```

Trailing closures make code **cleaner** and **more readable**, especially in SwiftUI or async tasks.

How do Closures Capture Values?

Definition:

Closures **capture constants and variables** from the surrounding context. Even if those variables go out of scope, the closure **retains** them.

Example:

```
func makeCounter() -> () -> Int {
    var count = 0
    return {
        count += 1
        return count
    }
}

let counter = makeCounter()
print(counter()) // 1
print(counter()) // 2
```

The closure captures `count` and keeps it alive across multiple calls.

What is a Retain Cycle in Closures, and How to Avoid It?

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Definition:

A **retain cycle** occurs when two objects (e.g., a class and a closure) **strongly reference** each other, preventing deallocation and causing a **memory leak**.

Example Problem:

```
class ViewModel {
    var message = "Hi"
    var handler: (() -> Void)?

    func setup() {
        handler = {
            print(self.message) // ❌ Causes strong reference cycle
        }
    }
}
```

Solution: Use `[weak self]` or `[unowned self]`

```
func setup() {
    handler = { [weak self] in
        print(self?.message ?? "")
    }
}
```

- `weak` makes `self` an optional and avoids a strong reference.
- Use `unowned` if you're sure `self` will never be `nil` (not recommended unless you're certain).

Summary Table:

Concept	Description	Example

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Closure	Reusable block of code	`{ name in "Hello, \(name)" }`
Trailing Closure	Closure passed outside function parentheses	`someFunction { ... }`
Capture Values	Closure remembers context variables	Capturing `count` in a counter
Retain Cycle	Strong references between closure & class causing memory leak	`self` inside closure
Avoiding Retain Cycle	Use `[weak self]` or `[unowned self]`	`handler = { [weak self] in ... }`

Explain how ARC works in Swift

Definition:

ARC (Automatic Reference Counting) is a memory management system used by Swift to **automatically track and manage** the memory usage of class instances. It ensures that objects are kept in memory **as long as they're needed**, and automatically deallocated when **no references** to them remain.

How ARC Works:

- Every time you assign a class instance to a variable, constant, or property, ARC increases the reference count.
- When the reference is removed, the count decreases.
- When the count hits **zero**, the memory is **freed automatically**.

Example:

```
class Person {
    var name: String
    init(name: String) {
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
        self.name = name
        print("\(name) is initialized")
    }
    deinit {
        print("\(name) is deallocated")
    }
}

var person1: Person? = Person(name: "Ruchit")
person1 = nil // Reference count drops to 0 — instance deallocated
```

What's the difference between strong, weak, and unowned references?

Reference Types Explained:

Reference Type	Retains Object?	Optional ?	Use Case
strong	Yes	❌ No	Default, retains memory
weak	❌ No	Yes	To avoid retain cycles, when object can be nil
unowned	❌ No	❌ No	Avoids retain cycles, when object is guaranteed to exist

---

Example with strong (default):

```
class Dog {
    var name: String
    init(name: String) { self.name = name }
}

class Owner {
    var dog: Dog // strong reference by default
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
}
```

Example with weak:

```
class Dog {
    var name: String
    init(name: String) { self.name = name }
}

class Owner {
    weak var dog: Dog? // doesn't increase ref count
}
```

How does ARC handle cyclic references in closures and class instances?

Retain Cycle (Reference Cycle):

Occurs when **two class instances strongly reference each other**, so neither is deallocated, causing a **memory leak**.

Class-to-Class Retain Cycle:

```
class A {
    var b: B?
}

class B {
    var a: A?
}

var objA: A? = A()
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
var objB: B? = B()
objA?.b = objB
objB?.a = objA

objA = nil
objB = nil // ❌ Neither gets deallocated — memory leak!
```

Fix using **weak** or **unowned**:

```
class B {
    weak var a: A? //  Prevents retain cycle
}
```

Closure Retain Cycle Example:

```
class ViewModel {
    var name = "Ruchit"
    var printName: (() -> Void)?

    func setup() {
        printName = {
            print(self.name) // ❌ `self` strongly captured → retain
cycle
        }
    }
}
```

Fix with **[weak self]**:

```
func setup() {
    printName = { [weak self] in
        print(self?.name ?? "")
    }
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
}
```

When would you use unowned instead of weak?

When to Use	Use weak	Use unowned
Object may be nil	Yes	❌ No (crashes if deallocated)
Object guaranteed to exist	❌ Optional	Safer, no optional unwrapping

---

Real-world Example: unowned in closures

```
class ViewController {
    var title = "Main"

    func doSomething() {
        performAction {
            print(self.title) // Retain cycle if not careful
        }
    }

    func performAction(_ completion: @escaping () -> Void) {
        completion()
    }
}

// Better:
performAction { [unowned self] in
    print(title) // `self` won't be retained, no optional
}
```

> ⚠️ Use `unowned` only when you're **100% sure** `self` will exist at execution time. If there's a chance it's been deallocated, use `weak`.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Summary:

Term	Description
ARC	Automatically manages memory by keeping track of strong references
strong	Default reference, retains the object
weak	Doesn't retain, used to avoid retain cycles with optional value
unowned	Doesn't retain, non-optional, assumes object always exists
Retain Cycle	Two objects holding strong refs to each other (or closure capturing `self`)

What are the SOLID Principles?

SOLID is an acronym for 5 design principles that make software more maintainable, scalable, and testable. It was introduced by Robert C. Martin (Uncle Bob).

Principle	Purpose	Keyword
S	Single Responsibility	One job per class
O	Open/Closed	Extend, don't modify
L	Liskov Substitution	Subtype must behave like supertype
I	Interface Segregation	Prefer small, role-specific protocols
D	Dependency Inversion	Depend on abstractions, not concrete types

---

Single Responsibility Principle (SRP)

Definition:

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

A class should **have only one reason to change**, meaning it should do **only one thing**.

Bad ViewController:

```
class UserProfileViewController: UIViewController {
    func fetchUserData() { /* API logic here */ }
    func parseJSON() { /* JSON parsing */ }
    func updateUI() { /* UI logic */ }
}
```

❌ This ViewController is handling networking, parsing, and UI = Too many responsibilities.

Refactored using SRP:

```
class UserService {
    func fetchUser(completion: @escaping (User) -> Void) { /* API Call
*/ }
}

class UserProfileViewModel {
    var user: User?
    let service = UserService()
}

class UserProfileViewController: UIViewController {
    var viewModel = UserProfileViewModel()
    func updateUI() { /* UI rendering only */ }
}
```

Each class handles **one responsibility**: networking, data holding, and UI.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Open/Closed Principle (OCP)

Definition:

Software entities (classes, functions) should be **open for extension**, but **closed for modification**.

Example using Strategy Pattern:

```
protocol PaymentMethod {
    func pay(amount: Double)
}

class CreditCardPayment: PaymentMethod {
    func pay(amount: Double) { print("Paid with credit card") }
}

class PayPalPayment: PaymentMethod {
    func pay(amount: Double) { print("Paid with PayPal") }
}

class PaymentProcessor {
    var method: PaymentMethod
    init(method: PaymentMethod) {
        self.method = method
    }

    func processPayment(amount: Double) {
        method.pay(amount: amount)
    }
}
```

> You can add new payment types (e.g., ApplePay) **without modifying** existing logic.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Liskov Substitution Principle (LSP)

Definition:

Objects of a subclass should be **substitutable** for objects of their superclass **without breaking the behavior**.

Violation Example:

```
class Bird {
    func fly() {}
}

class Penguin: Bird {
    override func fly() {
        // Penguins can't fly – wrong behavior
    }
}
```

❌ Penguin cannot behave like Bird. It breaks the contract.

Refactor:

```
protocol Bird { }
protocol FlyingBird: Bird {
    func fly()
}

class Eagle: FlyingBird {
    func fly() { print("Eagle flies") }
}

class Penguin: Bird { }
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Now, only `FlyingBird` conforms to `fly()`, respecting LSP.

Interface Segregation Principle (ISP)

Definition:

Prefer many **small, specific protocols** over one large, general-purpose protocol.

Bad Example:

```
protocol Vehicle {
    func drive()
    func fly()
}

class Car: Vehicle {
    func drive() {}
    func fly() {} // ❌ Irrelevant to Car
}
```

Better Design:

```
protocol Drivable {
    func drive()
}

protocol Flyable {
    func fly()
}

class Car: Drivable {
    func drive() {}
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
}
```

Classes now **only implement what they need**.

Dependency Inversion Principle (DIP)

Definition:

High-level modules should **not depend on low-level modules**. Both should depend on **abstractions**.

Tightly coupled:

```
class UserManager {
    let db = FirebaseManager() // hard-coded dependency
}
```

Decoupled via abstraction:

```
protocol DatabaseManager {
    func save(data: String)
}

class FirebaseManager: DatabaseManager {
    func save(data: String) { /* Firebase code */ }
}

class UserManager {
    let db: DatabaseManager
    init(db: DatabaseManager) {
        self.db = db
    }
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
    func saveUser() {
        db.save(data: "User")
    }
}
```

> You can now inject any `DatabaseManager` (e.g., CoreData, REST) during testing or runtime.

Applying SOLID in iOS

Principle	Where to Use in iOS
SRP	Separate networking, parsing, and UI in ViewControllers
OCP	Add features with protocols/extensions (e.g., payments, themes)
LSP	In protocol-oriented programming with reusable components
ISP	Modular protocol design (e.g., `UITableViewDataSource`, `UITableViewDelegate`)
DIP	Use protocol-based architecture for services, coordinators, and managers

---

All 5 SOLID principles with clean examples

```
// MARK: - 1. Single Responsibility Principle (SRP)

class UserService {
    func fetchUser(completion: @escaping (String) -> Void) {
        completion("Ruchit")
    }
}

class UserProfileViewModel {
    var userName: String?
    let service = UserService()
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
    func loadUser() {
        service.fetchUser { [weak self] name in
            self?.userName = name
        }
    }
}

class UserProfileViewController {
    let viewModel = UserProfileViewModel()

    func displayUser() {
        viewModel.loadUser()
        print("User: \(viewModel.userName ?? "")")
    }
}


// MARK: - 2. Open/Closed Principle (OCP)

protocol PaymentMethod {
    func pay(amount: Double)
}

class CreditCardPayment: PaymentMethod {
    func pay(amount: Double) {
        print("Paid \(amount) with Credit Card")
    }
}

class PayPalPayment: PaymentMethod {
    func pay(amount: Double) {
        print("Paid \(amount) with PayPal")
    }
}

class PaymentProcessor {
    var method: PaymentMethod

    init(method: PaymentMethod) {
        self.method = method
    }

    func process(amount: Double) {
```

```
        method.pay(amount: amount)
    }
}


// MARK: - 3. Liskov Substitution Principle (LSP)

protocol Bird {}
protocol FlyingBird: Bird {
    func fly()
}

class Eagle: FlyingBird {
    func fly() {
        print("Eagle soars high")
    }
}

class Penguin: Bird {} // Can't fly, so doesn't implement fly()


// MARK: - 4. Interface Segregation Principle (ISP)

protocol Drivable {
    func drive()
}

protocol Flyable {
    func fly()
}

class Car: Drivable {
    func drive() {
        print("Driving a car")
    }
}

class Plane: Flyable {
    func fly() {
        print("Flying a plane")
    }
}
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
// MARK: - 5. Dependency Inversion Principle (DIP)

protocol DatabaseManager {
    func save(data: String)
}

class FirebaseManager: DatabaseManager {
    func save(data: String) {
        print("Saved to Firebase: \(data)")
    }
}

class UserManager {
    let db: DatabaseManager

    init(db: DatabaseManager) {
        self.db = db
    }

    func saveUser(name: String) {
        db.save(data: name)
    }
}
```

SwiftUI Data Flow with `@State`, `@Binding`, `@ObservedObject`, etc.

In SwiftUI, data flows between views using property wrappers that help manage state and bindings.

`@State`: Used to store local state within a view. It is typically used for simple, mutable data like toggling a boolean or storing user input.
Example:

```
struct ContentView: View {
    @State private var isOn = false
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
    var body: some View {
        Toggle("Switch", isOn: $isOn)
    }
}
```

- Here, `isOn` is a local state variable, and `$isOn` is a binding to that state.

`@Binding`: Used to create a two-way connection to state data, typically passed from a parent view to a child. It allows the child view to modify the state in the parent view.
Example:

```
struct ParentView: View {
    @State private var isToggled = false

    var body: some View {
        ChildView(isOn: $isToggled)
    }
}

struct ChildView: View {
    @Binding var isOn: Bool

    var body: some View {
        Toggle("Child Switch", isOn: $isOn)
    }
}
```

- Here, the `ChildView` has access to the `isToggled` state from `ParentView` via a binding.

`@ObservedObject`: Used for observing a reference type object that conforms to the `ObservableObject` protocol. It notifies the view when the data in the object changes.
Example:

```
class UserSettings: ObservableObject {
    @Published var username: String = "John Doe"
}

struct ContentView: View {
```

```
    @ObservedObject var userSettings = UserSettings()

    var body: some View {
        Text(userSettings.username)
    }
}
```

- Here, `ContentView` observes changes in the `UserSettings` object.

`@EnvironmentObject`: A shared data object that can be accessed by any view in the hierarchy without passing it explicitly.
Example:

```
class AppSettings: ObservableObject {
    @Published var theme: String = "Light"
}

struct ContentView: View {
    @EnvironmentObject var settings: AppSettings

    var body: some View {
        Text("Current theme: \(settings.theme)")
    }
}
```

Handling Conditional Rendering in SwiftUI

In SwiftUI, conditional rendering is handled using `if`, `else`, and `switch` statements directly in the body of the view. This allows rendering views conditionally based on certain conditions or states.

Example:

```
struct ContentView: View {
    @State private var isLoggedIn = false

    var body: some View {
        VStack {
            if isLoggedIn {
```

```
                Text("Welcome back!")
            } else {
                Text("Please log in.")
            }
            Button(action: {
                isLoggedIn.toggle()
            }) {
                Text(isLoggedIn ? "Log Out" : "Log In")
            }
        }
    }
}
```

In this example, the view conditionally renders either a welcome message or a login prompt based on the `isLoggedIn`state.

View Updating: SwiftUI vs. UIKit

- **SwiftUI**: The view hierarchy in SwiftUI is declarative, meaning the views automatically update when the state changes. SwiftUI observes the state and re-renders only the parts of the view hierarchy that depend on the state. Views are lightweight and reactively updated based on the underlying data model changes.
  Example: SwiftUI automatically updates the UI when a `@State` or `@ObservedObject` changes.
- **UIKit**: UIKit follows an imperative approach where you must explicitly update the UI when the model changes. You might need to call `setNeedsLayout()` or `reloadData()` to trigger a view update.
  Example: In UIKit, you often need to explicitly tell the view to refresh after a data change (e.g., using `tableView.reloadData()` for a `UITableView`).

Building a Custom ViewModifier

A `ViewModifier` is used to apply changes to a view, such as styling or transformations, in a reusable manner.

Example:

```
struct RedBorderModifier: ViewModifier {
    func body(content: Content) -> some View {
```

```
        content
            .border(Color.red, width: 5)
            .padding()
    }
}

extension View {
    func redBorder() -> some View {
        self.modifier(RedBorderModifier())
    }
}

struct ContentView: View {
    var body: some View {
        Text("Hello, World!")
            .redBorder()  // Applying the custom modifier
    }
}
```

What is a Property Wrapper in Swift?

A **property wrapper** in Swift is a generic structure or class that encapsulates the behavior for a property. It provides a way to add logic around property access (getters and setters) while keeping the property declaration clean and reusable. Property wrappers can be used for behaviors like validation, data storage, and synchronization.

A property wrapper is defined using the `@` symbol and can be applied to any property.

Example of a basic property wrapper:

```
@propertyWrapper
struct Uppercase {
    private var value: String

    init(wrappedValue: String) {
        self.value = wrappedValue
    }
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
    var wrappedValue: String {
        get { value }
        set { value = newValue.uppercased() }
    }
}

struct User {
    @Uppercase var name: String
}

let user = User(name: "john doe")
print(user.name)  // Outputs: "JOHN DOE"
```

In this example, the `@Uppercase` property wrapper ensures that the `name` property is always stored in uppercase.

How does `@State` or `@AppStorage` work under the hood?

Both `@State` and `@AppStorage` are specialized property wrappers provided by SwiftUI to handle data binding, persistence, and UI updates.

`@State`: Manages mutable state for a view. When the state changes, SwiftUI re-renders the view. It's used for local state management within a view.
Under the hood: `@State` works by storing a value in a private, mutable property that is only accessible by the view itself. When the value changes, SwiftUI triggers a re-render of the view that uses this state. The value is stored in memory and is not persisted across app launches.
Example:

```
struct ContentView: View {
    @State private var counter = 0

    var body: some View {
        VStack {
            Text("Counter: \(counter)")
            Button("Increment") {
                counter += 1
            }
```

```
        }
    }
}
```

- Here, the counter value is managed locally within the view using `@State`, and the view updates automatically when the counter changes.

`@AppStorage`: Provides a way to store and retrieve data from the `UserDefaults` system with automatic synchronization. It's useful for persisting simple data like user settings.
Under the hood: `@AppStorage` wraps `UserDefaults` and binds a property to a value in `UserDefaults`. Any changes to the property are automatically stored in `UserDefaults`, and the view is re-rendered if the value changes.
Example:

```
struct ContentView: View {
    @AppStorage("userName") private var userName: String = "Guest"

    var body: some View {
        Text("Hello, \(userName)!")
    }
}
```

- Here, the `userName` property is bound to the `UserDefaults` value for `"userName"`. Any changes to `userName` will be persisted across app launches.

Have you created a custom property wrapper? What was the use case?

Yes, custom property wrappers are useful when you want to encapsulate logic for a property, such as validation, formatting, or caching.

Example Use Case: Validating a string for email format

```
@propertyWrapper
struct ValidEmail {
    private var email: String

    init(wrappedValue: String) {
        self.email = wrappedValue
    }
```

```
    var wrappedValue: String {
        get { email }
        set {
            // Simple email validation (for demonstration purposes)
            if newValue.contains("@") {
                email = newValue
            } else {
                email = "Invalid email"
            }
        }
    }
}

struct User {
    @ValidEmail var email: String
}

let user = User(email: "john.doe@example.com")
print(user.email)  // Outputs: "john.doe@example.com"
```

In this case, the `@ValidEmail` property wrapper checks whether the provided email contains an "@" symbol and updates the email only if valid. Otherwise, it assigns "Invalid email" as the value.

How do property wrappers behave with Codable?

Property wrappers can work with `Codable`, but their behavior must be managed carefully. Since `Codable` requires properties to be encoded and decoded, you need to define how the wrapper interacts with the encoding and decoding process.

Example: Using `@State` with `Codable`

If you use a property wrapper like `@State`, you don't typically encode the state itself (since it's specific to the view lifecycle). However, with custom property wrappers, you can implement encoding and decoding manually to make them work with `Codable`.

Example with a custom `Codable` property wrapper:

```
@propertyWrapper
struct CodableString: Codable {
    var value: String

    init(wrappedValue: String) {
        self.value = wrappedValue
    }

    var wrappedValue: String {
        get { value }
        set { value = newValue }
    }
}

struct User: Codable {
    @CodableString var name: String
}

let user = User(name: "John Doe")

// Encoding
let encoder = JSONEncoder()
if let encoded = try? encoder.encode(user) {
    print(String(data: encoded, encoding: .utf8)!)  // Outputs:
{"name":"John Doe"}
}

// Decoding
let decoder = JSONDecoder()
if let decoded = try? decoder.decode(User.self, from: encoded) {
    print(decoded.name)  // Outputs: "John Doe"
}
```

In this example, the `@CodableString` custom wrapper automatically integrates with the `Codable` protocol. The `wrappedValue` is used during encoding and decoding, ensuring that the property behaves correctly.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

How do you handle asynchronous code using `async/await` in Swift?

In Swift, **`async/await`** is a language feature that simplifies working with asynchronous code by making it more readable and structured. It allows you to write asynchronous code in a way that looks synchronous, making it easier to reason about.

- **`async`**: Marks a function or closure as asynchronous. It indicates that the function will perform some work that doesn't complete immediately (e.g., network requests, file I/O).
- **`await`**: Pauses the execution of the current function until the asynchronous operation completes. You can only call `await` inside `async` functions.

Example:

```
import Foundation

// Async function simulating a network request
func fetchData() async -> String {
    // Simulating network delay
    await Task.sleep(2 * 1_000_000_000)  // 2 seconds
    return "Data fetched"
}

// Calling the async function
func loadData() async {
    let data = await fetchData()
    print(data)
}

Task {
    await loadData()
}
```

Here, `fetchData()` is an asynchronous function, and `loadData()` awaits its result. The `Task` closure allows running the asynchronous code in an appropriate context (outside the main thread).

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

What is structured concurrency?

Structured concurrency is a model that organizes the execution of concurrent tasks in a structured way, meaning that tasks are given a clear scope and lifetime, and their execution is explicitly managed. In Swift, this means that you control when tasks are created, when they should be awaited, and when they are canceled, ensuring that tasks don't outlive their intended scope.

Swift 5.5 introduces structured concurrency through features like `Task` and `TaskGroup`. This model ensures that tasks are created within a well-defined scope, making it easier to manage their lifecycle and avoid issues like unintentional task leakage.

Example of structured concurrency:

```
func fetchMultipleData() async {
    async let data1 = fetchData()
    async let data2 = fetchData()

    // Await both concurrently
    let results = await [data1, data2]
    print(results)
}
```

Here, the `async let` syntax runs both `fetchData()` calls concurrently but ensures they are awaited together at the end, maintaining structured concurrency.

How would you convert a completion handler-based API to `async/await`?

You can convert a **completion handler-based API** into an `async/await` API using a helper function that wraps the existing completion handler-based method in an `async` function.

Example:

Suppose you have an old API that uses a completion handler:

```
func fetchDataWithCompletion(completion: @escaping (String?, Error?)
-> Void) {
    // Simulate a network request
    DispatchQueue.global().async {
```

```
        // Simulate delay
        sleep(2)
        completion("Data fetched", nil)
    }
}
```

You can wrap this API in an `async` function like this:

```
func fetchData() async throws -> String {
    return try await withCheckedThrowingContinuation { continuation in
        fetchDataWithCompletion { data, error in
            if let error = error {
                continuation.resume(throwing: error)
            } else if let data = data {
                continuation.resume(returning: data)
            }
        }
    }
}
```

In this example, the `withCheckedThrowingContinuation` function bridges the completion handler-based API to `async/await`. The `fetchData()` function now uses `async/await` instead of callbacks, making it easier to use in modern Swift code.

What's the difference between `Task`, `DetachedTask`, and `TaskGroup`?

In Swift concurrency, `Task`, `DetachedTask`, and `TaskGroup` are all used to create concurrent tasks, but they have different use cases and behaviors.

> **`Task`**: Represents a unit of asynchronous work tied to the current context. It is automatically canceled when the scope in which it was created ends (structured concurrency). This is typically used for tasks that need to run concurrently but should follow the lifecycle of the function or scope they are created in.
> **Example**:
>
> ```
> Task {
>     let result = await fetchData()
> ```

```
    print(result)
}
```

- **`DetachedTask`**: Represents a unit of asynchronous work that is **not tied** to the current context or scope. It runs independently and is not automatically canceled when the function or scope that created it ends. This is useful for long-running tasks that should not be canceled automatically.
  Example:

```
    DetachedTask {

    let result = await fetchData()
    print(result)
}
```

- **`TaskGroup`**: A way to run multiple asynchronous tasks concurrently and wait for all of them to finish. `TaskGroup`allows for structured concurrency while managing multiple concurrent tasks. You can add tasks to a group and await their results in a structured and controlled manner.
  Example:

```
    func fetchMultipleData() async {

    await withTaskGroup(of: String.self) { group in
        group.async {
            return await fetchData()
        }
        group.async {
            return await fetchData()
        }

        for await result in group {
            print(result)
        }
    }
}
```

- Here, `withTaskGroup` creates a group of tasks. The tasks are run concurrently, and the results are awaited using `for await`.

Summary of Differences:

- **`Task`**: Tied to the current context and will be canceled when the context ends. Suitable for tasks that should follow the scope's lifecycle.
- **`DetachedTask`**: Runs independently from the current context. Ideal for long-running tasks that shouldn't be automatically canceled.
- **`TaskGroup`**: A way to manage multiple concurrent tasks. Provides structured concurrency, ensuring that all tasks in the group are completed before moving forward.

What is Copy-on-Write in Swift? How does it optimize memory usage?

Copy-on-Write (COW) is a memory optimization technique used to avoid unnecessary copying of data until it is actually modified. In Swift, data structures like `Array`, `String`, and `Dictionary` implement COW to ensure that copying only happens when it is needed (i.e., when the data is modified).

This optimization helps reduce memory usage and improve performance, as it delays the actual duplication of data until the original or copied structure is mutated.

How COW Works:

- **When copying**: The system doesn't actually copy the underlying data; it just shares the same reference to the data.
- **When modifying**: The data is copied only if one of the references attempts to modify the data, ensuring that changes are isolated to that reference.

This allows multiple instances of a collection (like an array) to share the same memory until one of them changes, at which point a true copy is made to avoid affecting the other instances.

2. How does Swift manage memory when copying `Array`, `String`, or `Dictionary`?

- **Arrays**: Swift's `Array` type uses COW, meaning that when you assign an array to a new variable or pass it to a function, Swift doesn't immediately copy the array. Only when the new array is modified does Swift make a copy of the data, ensuring that the original array remains unchanged.
- **Strings**: `String` in Swift also uses COW. When you assign a string to another variable or pass it into a function, no actual copy happens. However, if either the original string or the new string is modified, a new copy of the string is created for that variable.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

- **Dictionaries**: Like `Array` and `String`, `Dictionary` in Swift uses COW. When a dictionary is copied, the underlying storage is shared between the original and the copy until one of them is modified.

Example Demonstrating Copy-on-Write with `Array`:

```
var array1 = [1, 2, 3]
var array2 = array1  // No copy yet, array1 and array2 share the same memory.

array2.append(4)  // This will trigger a copy because we are modifying array2.

print(array1)  // [1, 2, 3]
print(array2)  // [1, 2, 3, 4]
```

In this example:

- `array1` is copied to `array2`, but no actual copy happens yet; they share the same memory.
- When `array2` is modified by appending an element, Swift creates a new copy of the array for `array2`. The original `array1` remains unchanged.

How would you demonstrate Copy-on-Write behavior with a custom struct?

You can implement a custom struct that behaves like COW by manually managing the underlying data storage. You would need to use a reference type (like `class`) inside the struct to manage the actual data and ensure that the copy happens only when the data is mutated.

Example of COW with a Custom Struct:

```
class DataStorage {
    var data: [Int]

    init(data: [Int]) {
        self.data = data
    }
}
```

```
struct CopyOnWriteStruct {
    private var storage: DataStorage

    init(data: [Int]) {
        self.storage = DataStorage(data: data)
    }

    // This computed property is used to ensure a unique copy when
modifying.
    private mutating func copyIfNeeded() {
        if !isKnownUniquelyReferenced(&storage) {
            storage = DataStorage(data: storage.data)  // Make a copy
only if necessary.
        }
    }

    var data: [Int] {
        get { storage.data }
        set {
            copyIfNeeded()
            storage.data = newValue
        }
    }
}

var struct1 = CopyOnWriteStruct(data: [1, 2, 3])
var struct2 = struct1  // No copy yet, struct1 and struct2 share the
same data.

struct2.data.append(4)  // This triggers a copy when struct2 is
modified.

print(struct1.data)  // [1, 2, 3]
print(struct2.data)  // [1, 2, 3, 4]
```

In this custom struct:

- `DataStorage` is a class that holds the actual data. This class is used to share data between instances of `CopyOnWriteStruct`.
- The `copyIfNeeded()` method ensures that a copy of the data is made only if the struct is not uniquely referenced. If the data is being modified, it makes a copy of the data, following the COW principle.
- The `data` property is where we read and write the data, ensuring that the data is copied only when necessary (i.e., when mutation occurs).

Summary:

- **Copy-on-Write (COW)** is a technique to optimize memory usage by delaying data duplication until necessary.
- Swift's `Array`, `String`, and `Dictionary` types use COW, meaning they share memory until one of the references modifies the data.
- You can demonstrate COW behavior in custom structs by using reference types for storage and copying the data only when mutation happens.

What's the difference between a shallow and a deep copy?

- **Shallow Copy**: A shallow copy of an object creates a new reference to the same underlying data. In essence, it copies the references, but not the actual data. Any changes made to the original data will affect the shallow copy and vice versa, because both the original and the copy are referencing the same memory.
- **Deep Copy**: A deep copy creates a completely independent copy of the data, including all objects or references contained within the original object. It duplicates the entire structure, so changes to the original object will not affect the deep copy, and vice versa.

Example:

Consider an object with a reference type (class) inside another object:

```
class Car {
    var model: String
    var owner: String

    init(model: String, owner: String) {
        self.model = model
        self.owner = owner
    }
}
```

```
class Person {
    var name: String
    var car: Car

    init(name: String, car: Car) {
        self.name = name
        self.car = car
    }
}

let originalCar = Car(model: "Tesla", owner: "John")
let originalPerson = Person(name: "Alice", car: originalCar)

// Shallow copy
let shallowPerson = originalPerson
shallowPerson.car.owner = "Bob"  // Modifying car owner in shallow
copy

print(originalPerson.car.owner)  // Outputs: "Bob" (same car, both
refer to the same instance)

// Deep copy
let deepPerson = Person(name: originalPerson.name, car: Car(model:
originalPerson.car.model, owner: originalPerson.car.owner))
deepPerson.car.owner = "Charlie"  // Changing deep copy's car owner

print(originalPerson.car.owner)  // Outputs: "Bob" (original is
unaffected)
print(deepPerson.car.owner)      // Outputs: "Charlie" (independent
copy)
```

- **Shallow copy**: `shallowPerson` is just a new reference to `originalPerson`. When you modify `shallowPerson.car.owner`, it affects `originalPerson` because they both share the same `Car` object.
- **Deep copy**: `deepPerson` has a new `Car` object that copies the properties of `originalPerson.car`, so modifying `deepPerson.car.owner` does not affect `originalPerson`.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

How does this apply in Swift with structs vs classes?

- **Structs**: In Swift, **structs are value types**, meaning they are copied by value. When you assign a struct to a new variable or pass it into a function, a **deep copy** is created automatically. Each instance of a struct holds its own copy of the data, so modifying one struct does not affect another.
- **Classes**: In contrast, **classes are reference types**, meaning they are copied by reference. When you assign a class to a new variable or pass it into a function, only the reference (or memory address) is copied, not the actual data. Both the original object and the copied reference point to the same data. Therefore, changes made to one instance will affect the other.

Example with Struct (Deep Copy by Default):

```
struct Rectangle {
    var width: Int
    var height: Int
}

var rect1 = Rectangle(width: 10, height: 20)
var rect2 = rect1  // This creates a deep copy, as structs are value
types
rect2.width = 15    // Modifying rect2 does not affect rect1

print(rect1.width)  // Outputs: 10
print(rect2.width)  // Outputs: 15
```

In this case, because structs are value types, when `rect1` is assigned to `rect2`, a **deep copy** occurs by default, and changes to `rect2` do not affect `rect1`.

Example with Class (Shallow Copy by Default):

```
class Rectangle {
    var width: Int
    var height: Int

    init(width: Int, height: Int) {
        self.width = width
        self.height = height
```

```
    }
}

var rect1 = Rectangle(width: 10, height: 20)
var rect2 = rect1  // rect2 is a reference to rect1, not a copy

rect2.width = 15    // Modifying rect2 also affects rect1, since they
share the same object

print(rect1.width)  // Outputs: 15
print(rect2.width)  // Outputs: 15
```

Here, because `Rectangle` is a class (reference type), both `rect1` and `rect2` refer to the same object in memory. Modifying one instance (`rect2`) also modifies the other (`rect1`), demonstrating a **shallow copy**.

How do you implement a deep copy of a class?

To implement a deep copy of a class, you need to manually create a copy of each property or reference inside the class, especially if the class contains other objects or reference types. This can be done by conforming to the `NSCopying`protocol or by implementing a custom `copy()` method.

Example of Deep Copy with a Class:

```
class Car {
    var model: String
    var owner: String

    init(model: String, owner: String) {
        self.model = model
        self.owner = owner
    }

    // Implementing deep copy
    func copy() -> Car {
        return Car(model: self.model, owner: self.owner)
    }
```

```
}

class Person {
    var name: String
    var car: Car

    init(name: String, car: Car) {
        self.name = name
        self.car = car
    }

    // Implementing deep copy for Person
    func copy() -> Person {
        let carCopy = self.car.copy()  // Deep copy the car
        return Person(name: self.name, car: carCopy)
    }
}

let originalCar = Car(model: "Tesla", owner: "John")
let originalPerson = Person(name: "Alice", car: originalCar)

let copiedPerson = originalPerson.copy()  // Deep copy of the person
copiedPerson.car.owner = "Bob"  // Modify the copied person's car
owner

print(originalPerson.car.owner)  // Outputs: "John" (unchanged)
print(copiedPerson.car.owner)    // Outputs: "Bob" (modified in the
deep copy)
```

In this example:

- The `Car` class has a `copy()` method that creates a new `Car` object with the same properties.
- The `Person` class has a `copy()` method that creates a new `Person` object with a deep copy of the `car` property.

By implementing custom `copy()` methods, you ensure that the objects are fully copied, including any reference types they contain, and modifications to the copy do not affect the original.

Summary:

- **Shallow copy** creates a new reference to the same object, and changes to the object affect both references.
- **Deep copy** creates a completely independent copy of the object and its contents, with no shared references.
- **Structs** in Swift are value types and automatically perform deep copies, whereas **classes** are reference types and perform shallow copies by default.
- To implement deep copying in classes, you can manually create copies of properties or implement a custom `copy()`method.

What is the `Copyable` protocol introduced in Swift?

In **Swift 5.9**, the `Copyable` protocol was introduced to provide a standardized way to make **reference types** (like classes) behave more like **value types** by enabling them to perform a **deep copy** when required. The protocol is specifically designed to enable "value semantics" for reference types, allowing objects to be copied independently without changing the original data.

- **Value semantics** means that when a variable or constant is copied, its data is also copied, and changes to the copy do not affect the original.
- **Reference semantics** means that when a variable or constant is copied, both refer to the same memory location, so changes to the copy will affect the original.

The `Copyable` protocol allows reference types to opt into a behavior that mimics value types (i.e., they can be copied fully, not just by reference).

Key Characteristics of `Copyable`:

- It ensures that a deep copy of an object is created when necessary.
- It helps manage the copying of complex objects like classes, which traditionally use reference semantics.
- It can help avoid common issues related to reference types when working with copies.

How does it help with value semantics in reference types?

By conforming to the `Copyable` protocol, **reference types** (like `class`) can be copied by value, similar to how **value types** (like `struct` and `enum`) behave. This means that changes to the

copied object will not affect the original object, addressing a major distinction between reference types and value types in Swift.

In essence, it provides a way to apply **value semantics** to a class, which is typically a **reference type**. This is particularly useful when you need to ensure that modifying a copy of an object does not unintentionally modify other references to the same object.

Without `Copyable`, reference types are always shared by reference, so copying a class object usually results in both variables pointing to the same data. With `Copyable`, classes can provide their own logic for creating copies that ensure the original and copy are independent.

Can you give an example of implementing `Copyable` in your code?

Here's how you can implement the `Copyable` protocol in Swift:

Example:

```
// Define a class that conforms to the `Copyable` protocol
import Foundation

protocol Copyable {
    func copy() -> Self
}

class Person: Copyable {
    var name: String
    var age: Int

    init(name: String, age: Int) {
        self.name = name
        self.age = age
    }

    // Implement the `copy` method for deep copy
    func copy() -> Person {
        return Person(name: self.name, age: self.age)
    }
}
```

```
let originalPerson = Person(name: "Alice", age: 30)
let copiedPerson = originalPerson.copy()

// Modify the copy
copiedPerson.name = "Bob"
copiedPerson.age = 25

print(originalPerson.name)  // Outputs: "Alice" (unchanged)
print(originalPerson.age)   // Outputs: 30 (unchanged)
print(copiedPerson.name)    // Outputs: "Bob" (modified)
print(copiedPerson.age)     // Outputs: 25 (modified)
```

Explanation:

- We define the `Copyable` protocol with a `copy()` method.
- The `Person` class conforms to the `Copyable` protocol and implements the `copy()` method, which creates a new instance of `Person` with the same properties (thus creating a **deep copy**).
- When the `copiedPerson` is modified, the changes do not affect `originalPerson`, demonstrating the **value semantics**.

Summary of the Benefits:

- **Standardized copying**: The `Copyable` protocol provides a standard way to ensure that objects of reference types (like classes) are deeply copied, mimicking value types.
- **Value Semantics for Classes**: With `Copyable`, classes can be made to behave with value semantics, providing independent copies that are unaffected by changes to the original instance.
- **Cleaner Code**: It simplifies working with reference types when deep copying is needed, removing the need for custom copy logic in every class.

What's the difference between DispatchQueue.main.async and .sync?

`DispatchQueue.main.async`: This executes the block of code asynchronously on the main queue, meaning the code will be executed sometime in the future, but control returns to the calling thread immediately. This is non-blocking, allowing the main thread to continue running other tasks.
Example:

```
DispatchQueue.main.async {
    // This code will run asynchronously on the main queue
    print("This is async on the main queue")
}
// Control returns immediately
```

- **`DispatchQueue.main.sync`**: This executes the block of code synchronously on the main queue, meaning the calling thread is blocked until the task completes. If you call `.sync` on the main queue from the main thread itself, it will lead to a deadlock because the main thread is blocked waiting for itself to finish.
  Example:

```
DispatchQueue.main.sync {
    // This code will run synchronously on the main queue, blocking the calling thread
    print("This is sync on the main queue")
}
// Control returns after the task completes
```

- **Key difference**:
  - `.async`: Non-blocking, executes the code in the future without blocking the current thread.
  - `.sync`: Blocking, waits for the code to complete before returning to the caller.

When would you use serial vs concurrent queues?

Serial Queue: A serial queue processes tasks one at a time, in the order they were added. It's useful when the tasks depend on each other or need to be executed sequentially to avoid race conditions.
Example:

```
let serialQueue = DispatchQueue(label: "com.example.serialQueue")
serialQueue.async {
    print("Task 1")
}
serialQueue.async {
    print("Task 2")
}
```

```
// "Task 1" will complete before "Task 2" starts
```

- **Concurrent Queue**: A concurrent queue can execute multiple tasks simultaneously (concurrently). This is useful for tasks that can be performed in parallel and do not depend on each other.
  Example:

```
let concurrentQueue = DispatchQueue(label:
"com.example.concurrentQueue", attributes: .concurrent)

concurrentQueue.async {
    print("Task 1")
}
concurrentQueue.async {
    print("Task 2")
}
// "Task 1" and "Task 2" can run simultaneously
```

How do you avoid deadlocks when using GCD?

A deadlock happens when two threads are waiting on each other to complete, which results in an infinite wait. Common causes of deadlocks with GCD include calling `.sync` on the same queue from within that queue.

To avoid deadlocks:

- Avoid calling `DispatchQueue.main.sync` from the main thread.
- Avoid calling `.sync` on a queue that is already executing tasks on itself (e.g., calling `.sync` on the main queue from the main thread).

Example of a deadlock:

```
DispatchQueue.main.sync {
    DispatchQueue.main.sync {
        // This will cause a deadlock because the main thread is
blocked waiting for itself
    }
}
```

OperationQueue

An **`OperationQueue`** is a higher-level abstraction over GCD that allows you to manage and organize operations (tasks) in a more structured way, providing more control over execution.

What are the advantages of using OperationQueue over GCD?

- **Task Management**: `OperationQueue` allows better management of tasks, including prioritization, dependencies, and cancellation.
- **Execution Control**: You can control the maximum number of concurrent operations via the `maxConcurrentOperationCount` property.
- **Cancellation**: Operations can be canceled, whereas tasks in GCD cannot.

Dependencies: `OperationQueue` allows you to define dependencies between operations (i.e., operation A must finish before operation B starts).
Example:

```
let operationQueue = OperationQueue()

let op1 = BlockOperation {
    print("Task 1")
}

let op2 = BlockOperation {
    print("Task 2")
}

op2.addDependency(op1)  // op2 will start after op1 completes

operationQueue.addOperations([op1, op2], waitUntilFinished: false)
```

How do you cancel an operation mid-execution?

You can cancel an operation by calling its `cancel()` method. However, for this to be effective, the operation needs to regularly check for cancellation within its execution code and stop if canceled.

Example:

```
let operationQueue = OperationQueue()
```

```
let op = BlockOperation {
    for i in 0..<10 {
        if op.isCancelled {
            print("Operation was cancelled")
            return
        }
        print(i)
    }
}

operationQueue.addOperation(op)

DispatchQueue.global().asyncAfter(deadline: .now() + 2) {
    op.cancel()  // Cancel the operation after 2 seconds
}
```

In this example, the operation checks `isCancelled` inside its loop, and if the operation is canceled, it will stop executing further tasks.

How do you manage dependencies between operations?

You can define dependencies between operations using the `addDependency()` method. This ensures that one operation cannot start until another has completed.

Example:

```
let operationQueue = OperationQueue()

let op1 = BlockOperation {
    print("Task 1")
}

let op2 = BlockOperation {
    print("Task 2")
}

let op3 = BlockOperation {
    print("Task 3")
```

```
}

op2.addDependency(op1)  // op2 will run after op1 finishes
op3.addDependency(op2)  // op3 will run after op2 finishes

operationQueue.addOperations([op1, op2, op3], waitUntilFinished:
false)
```

In this example:

- `op2` will run only after `op1` finishes.
- `op3` will run only after `op2` finishes.
- The operations are executed in sequence (`op1 -> op2 -> op3`).

Summary:

- **GCD** allows you to manage concurrency with simple methods (`async` and `sync`) but requires manual handling of synchronization, deadlocks, and task organization.
- **OperationQueue** provides higher-level abstractions for managing concurrent operations, including cancellation, dependencies, and prioritization.
- **Serial queues** are used for sequential execution, while **concurrent queues** allow parallel execution.
- To avoid **deadlocks** in GCD, avoid using `sync` on the main queue or the queue you're currently executing on.

Difference between `class`, `struct`, and `enum` in Swift

Class: A `class` is a reference type, meaning when you assign it to a new variable or pass it into a function, you're passing a reference to the same instance, not a copy. It can inherit from other classes and supports deinitialization (`deinit`), which is useful for memory management. Example:

```
class Dog {
    var name: String
    init(name: String) {
        self.name = name
    }
}
let dog1 = Dog(name: "Buddy")
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
let dog2 = dog1
dog2.name = "Max"
print(dog1.name)  // Outputs: Max (dog1 and dog2 refer to the same
object)
```

- **Struct**: A `struct` is a value type, meaning when you assign it to a new variable or pass it into a function, it creates a copy of the instance. It does not support inheritance or deinitialization, but it has automatic synthesis for `init`, `==`(equality comparison), and more.
  Example:
  swift

```
struct Dog {

    var name: String
}
var dog1 = Dog(name: "Buddy")
var dog2 = dog1
dog2.name = "Max"
print(dog1.name)  // Outputs: Buddy (dog1 and dog2 are independent)
```

- **Enum**: An `enum` (enumeration) is a value type that defines a common type for a group of related values. You can associate values with cases, and enums are often used for state management and control flow.
  Example:
  swift

```
enum DogBreed {

    case bulldog, poodle, labrador
}
var myDogBreed = DogBreed.bulldog
myDogBreed = .poodle
```

What is ARC (Automatic Reference Counting)? How does memory management work in Swift?

ARC is a memory management system used by Swift to automatically keep track of the references to objects in memory. ARC ensures that memory is freed when an object is no longer being used, which helps prevent memory leaks.

How it works: Every time an object is assigned to a variable, a reference to that object is created. ARC counts how many references there are to an object. When an object's reference count reaches zero (i.e., there are no more strong references to it), ARC automatically deallocates the memory.
Example:

```
class Car {
    var brand: String
    init(brand: String) {
        self.brand = brand
    }
    deinit {
        print("\(brand) is being deallocated")
    }
}

var car1: Car? = Car(brand: "Tesla")
var car2 = car1  // Reference count = 2
car1 = nil  // Reference count = 1
car2 = nil  // Reference count = 0, "Tesla is being deallocated"
```

Strong, Weak, and Unowned References in Swift

Strong References: By default, all references to objects are strong, meaning they increase the reference count of the object. As long as there is a strong reference to an object, it will not be deallocated.
Example:

```
var dog = Dog(name: "Buddy")
// 'dog' is a strong reference to the Dog instance
```

- **Weak References**: A weak reference does not retain an object, meaning it does not increase the reference count. If an object is deallocated, weak references automatically become `nil`. Use weak references when there could be a cyclic reference (like between a parent and child), and you don't want one object to keep the other in memory.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Example:
swift

```
class Owner {
    var name: String
    init(name: String) {
        self.name = name
    }
}

class Dog {
    var owner: Owner?
}

var owner1: Owner? = Owner(name: "John")
var dog1 = Dog()
dog1.owner = owner1
owner1 = nil  // owner1 is deallocated, dog1.owner becomes nil (weak reference)
```

- **Unowned References**: An unowned reference is similar to a weak reference, but it assumes the referenced object will never be deallocated during the lifetime of the reference. Unowned references are used when there is a strong reference cycle, but you are confident the referenced object will be deallocated before the unowned reference.
  Example:
  swift

```
class Teacher {
    var name: String
    var student: Student?

    init(name: String) {
        self.name = name
    }
}

class Student {
```

```
    var name: String
    var teacher: Teacher?

    init(name: String) {
        self.name = name
    }
}

var teacher = Teacher(name: "Mr. Smith")
var student = Student(name: "Alice")

teacher.student = student
student.teacher = teacher  // Unowned reference to avoid strong
reference cycle
```

Value Types vs Reference Types in Swift

- **Value Types**: In Swift, value types are types where each instance keeps a unique copy of its data. When you assign a value type to another variable or pass it into a function, a **copy** of the original object is created. `struct`, `enum`, and basic types like `Int` are value types.
    - **Key characteristics**:
        - Copy on assignment.
        - Changes to one instance do not affect other instances.
        - Examples: `struct`, `enum`, `Int`, `Double`, `Array`, `Dictionary`, etc.

Example:

```
struct Point {
    var x: Int
    var y: Int
}
var point1 = Point(x: 1, y: 2)
var point2 = point1
point2.x = 10
print(point1.x)  // Outputs: 1
print(point2.x)  // Outputs: 10
```

- **Reference Types**: In Swift, reference types are types where instances share a reference to the same underlying data. When you assign a reference type to another variable or pass it into a function, **both variables point to the same object**. `class` is a reference type.
    - **Key characteristics**:
        - Shared references, not copies.
        - Changes to one reference affect all references to the object.
        - Examples: `class`, `closure`.

Example:

```
class Person {
    var name: String
    init(name: String) {
        self.name = name
    }
}

var person1 = Person(name: "John")
var person2 = person1
person2.name = "Steve"
print(person1.name)  // Outputs: Steve (both variables refer to the same instance)
```

Summary:

- **Class**: Reference type; supports inheritance and deinitialization.
- **Struct**: Value type; does not support inheritance; creates copies when assigned or passed.
- **Enum**: Value type; used to define related values.
- **ARC**: Automatically manages memory by counting references to objects; objects are deallocated when reference count reaches zero.
- **Strong references**: Keep objects in memory.
- **Weak references**: Do not keep objects in memory; automatically set to `nil` when the object is deallocated.
- **Unowned references**: Like weak references, but they assume the object will outlive the reference.
- **Value types** create independent copies, while **reference types** share a single object.

How does SwiftUI differ from UIKit?

- **Declarative vs. Imperative**:
    - **SwiftUI** is **declarative**, meaning you describe what the UI should look like and how it behaves based on the state. SwiftUI automatically updates the view when the state changes.
    - **UIKit** is **imperative**, meaning you explicitly tell the UI how to behave and manage state changes manually.
- **UI Construction**:
    - In **SwiftUI**, you define the layout using a combination of built-in views like `Text`, `Button`, `VStack`, `HStack`, etc., and modify the appearance by simply changing the state of the views.
    - In **UIKit**, you create UI components programmatically or through Storyboards and handle their lifecycle and interactions manually.
- **State Management**:
    - **SwiftUI** has built-in tools to manage state (`@State`, `@Binding`, `@ObservedObject`, `@EnvironmentObject`), which makes UI updates easier and more automatic.
    - In **UIKit**, you need to manually update the UI when state changes, using techniques like KVO (Key-Value Observing), `NSNotificationCenter`, or `Delegates`.
- **Code Reusability**:
    - **SwiftUI** makes code more reusable with **Views** as reusable components, simplifying UI construction and management of different screens.
    - **UIKit** also supports reuse, but it requires more boilerplate code, such as managing view controllers and complex navigation.

Data Flow in SwiftUI: `@State`, `@Binding`, `@ObservedObject`, `@EnvironmentObject`

- **`@State`**:
    - `@State` is used for **local** state management within a **view**. It's used to hold values that change and cause the view to update when the value changes.
    - **Scope**: The state is local to the view.

Example:

```
struct CounterView: View {
    @State private var count = 0

    var body: some View {
        VStack {
```

```
            Text("Count: \(count)")
            Button("Increment") {
                count += 1
            }
        }
    }
}
```

-
- **`@Binding`**:
    - `@Binding` allows you to **bind** a value to another view. It enables you to create a reference to a value that is owned by another view. When the value changes, it will reflect in both views.
    - **Scope**: Used for passing state data between parent and child views.

Example:

```
struct ParentView: View {
    @State private var isOn = false

    var body: some View {
        ToggleView(isOn: $isOn) // Binding the state to the child
    }
}

struct ToggleView: View {
    @Binding var isOn: Bool

    var body: some View {
        Toggle("Switch", isOn: $isOn)
    }
}
```

- **`@ObservedObject`**:
    - `@ObservedObject` is used for observing changes in an **external** state that conforms to the `ObservableObject`protocol. It's useful for sharing data across multiple views and automatically updating when the data changes.
    - **Scope**: Used for objects that are shared across multiple views and whose changes should trigger view updates.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Example:

```
class UserData: ObservableObject {
    @Published var name: String = "John Doe"
}

struct ProfileView: View {
    @ObservedObject var userData: UserData

    var body: some View {
        Text(userData.name)
    }
}
```

- **`@EnvironmentObject`**:
    - `@EnvironmentObject` is similar to `@ObservedObject` but is meant for data shared globally within an app (often set at a high level in the view hierarchy). It is automatically injected into the view environment and can be accessed from any child view in the hierarchy.
    - **Scope**: Used for global shared data, typically injected at the root of the view hierarchy.

Example:

```
class ThemeData: ObservableObject {
    @Published var themeColor: Color = .blue
}

struct ContentView: View {
    @EnvironmentObject var themeData: ThemeData

    var body: some View {
        Text("The theme color is \(themeData.themeColor.description)")
            .foregroundColor(themeData.themeColor)
    }
}
```

To inject the `@EnvironmentObject`:

```
@main
```

```
struct MyApp: App {
    @StateObject var themeData = ThemeData()

    var body: some Scene {
        WindowGroup {
            ContentView()
                .environmentObject(themeData)
        }
    }
}
```

How would you integrate a UIKit component in a SwiftUI view?

To integrate a UIKit component in SwiftUI, you can use **`UIViewRepresentable`** or **`UIViewControllerRepresentable`** to wrap UIKit components.

`UIViewRepresentable`: Used to wrap a UIKit view (like `UILabel`, `UIButton`, `UIImageView`, etc.) for use in SwiftUI.
Example:

```
struct UIKitButton: UIViewRepresentable {
    class Coordinator: NSObject {
        var action: () -> Void

        init(action: @escaping () -> Void) {
            self.action = action
        }
    }

    var title: String
    var action: () -> Void

    func makeUIView(context: Context) -> UIButton {
        let button = UIButton(type: .system)
        button.setTitle(title, for: .normal)
        button.addTarget(context.coordinator, action:
#selector(Coordinator.buttonTapped), for: .touchUpInside)
        return button
```

```
    }

    func updateUIView(_ uiView: UIButton, context: Context) {
        uiView.setTitle(title, for: .normal)
    }

    func makeCoordinator() -> Coordinator {
        return Coordinator(action: action)
    }
}

struct ContentView: View {
    var body: some View {
        UIKitButton(title: "Click Me") {
            print("Button tapped!")
        }
    }
}
```

`UIViewControllerRepresentable`: Used to wrap a UIKit view controller (like `UITableViewController`, `UIViewController`, etc.) for use in SwiftUI.
Example:

```
struct UIKitTableView: UIViewControllerRepresentable {
    class Coordinator: NSObject, UITableViewDelegate,
UITableViewDataSource {
        var items: [String]

        init(items: [String]) {
            self.items = items
        }

        func tableView(_ tableView: UITableView, numberOfRowsInSection
section: Int) -> Int {
            return items.count
        }
```

```
        func tableView(_ tableView: UITableView, cellForRowAt
indexPath: IndexPath) -> UITableViewCell {
            let cell = tableView.dequeueReusableCell(withIdentifier:
"Cell", for: indexPath)
            cell.textLabel?.text = items[indexPath.row]
            return cell
        }
    }

    var items: [String]

    func makeUIViewController(context: Context) ->
UITableViewController {
        let viewController = UITableViewController(style: .plain)
        viewController.tableView.delegate = context.coordinator
        viewController.tableView.dataSource = context.coordinator
        return viewController
    }

    func updateUIViewController(_ uiViewController:
UITableViewController, context: Context) {
        // Update the UITableView if needed
    }

    func makeCoordinator() -> Coordinator {
        return Coordinator(items: items)
    }
}

struct ContentView: View {
    var body: some View {
        UIKitTableView(items: ["Item 1", "Item 2", "Item 3"])
    }
}
```

Summary:

- **SwiftUI** differs from **UIKit** in its declarative nature, automatic state management, and view structure, whereas UIKit requires more manual management.
- **Data flow** in SwiftUI can be handled using property wrappers like `@State`, `@Binding`, `@ObservedObject`, and `@EnvironmentObject` for local and global state management.
- **UIKit components** can be integrated into SwiftUI views using **`UIViewRepresentable`** and **`UIViewControllerRepresentable`**.

Which architecture are you most comfortable with (MVVM, VIPER, Clean, etc.)?

I am most comfortable with **MVVM (Model-View-ViewModel)** and **Clean Architecture**. Both architectures provide a good balance between maintainability, testability, and separation of concerns.

- **MVVM** is highly suitable for SwiftUI because it provides a clear distinction between the UI layer (View) and the logic (ViewModel), which is ideal for data binding and reactive interfaces.
- **Clean Architecture** is preferred for complex applications where scalability, testability, and maintainability are crucial. It helps separate concerns into different layers (UI, Business Logic, Data) and keeps code decoupled.

Can you walk through how you structure a real-world app using MVVM?

In **MVVM (Model-View-ViewModel)**, we divide the app into three main components:

- **Model**: Represents the data and business logic of the application.
- **View**: Represents the UI and is responsible for displaying the data.
- **ViewModel**: Serves as the intermediary between the Model and View, holding the business logic for the View.

Here's an example of structuring a simple app using MVVM:

App Concept: A Simple Todo List App

- **Model**: Contains the data and logic for managing todos.
- **ViewModel**: Handles data manipulation and prepares data for display.
- **View**: Displays the UI to the user and binds to the ViewModel.

Step-by-Step Breakdown

Model:

```
struct Todo {
    var title: String
    var isCompleted: Bool
}

class TodoManager {
    private var todos: [Todo] = []

    func addTodo(title: String) {
        todos.append(Todo(title: title, isCompleted: false))
    }

    func getAllTodos() -> [Todo] {
        return todos
    }
}
```

ViewModel:

```
class TodoViewModel: ObservableObject {
    @Published var todos: [Todo] = []
    private var todoManager = TodoManager()

    func fetchTodos() {
        todos = todoManager.getAllTodos()
    }

    func addTodo(title: String) {
        todoManager.addTodo(title: title)
        fetchTodos()  // Refresh the data after adding a new todo
    }
}
```

View (SwiftUI):

```
struct TodoListView: View {
```

```
    @StateObject private var viewModel = TodoViewModel()
    @State private var newTodoTitle: String = ""

    var body: some View {
        VStack {
            TextField("Enter Todo", text: $newTodoTitle)
                .padding()
            Button(action: {
                viewModel.addTodo(title: newTodoTitle)
                newTodoTitle = ""
            }) {
                Text("Add Todo")
            }

            List(viewModel.todos) { todo in
                Text(todo.title)
            }
        }
        .onAppear {
            viewModel.fetchTodos()
        }
    }
}
```

Explanation:

- The **Model** represents the `Todo` structure and the `TodoManager` class, which holds the data and business logic (adding and fetching todos).
- The **ViewModel** (`TodoViewModel`) holds the app's business logic and acts as the data provider to the view. It's also responsible for refreshing the data after adding a todo.
- The **View** is a SwiftUI view that binds to the ViewModel. The UI is updated automatically when the data changes due to the use of `@Published` in the ViewModel.

Benefits of MVVM:

- **Separation of Concerns**: The ViewModel isolates logic and data from the view, making the view simpler and more focused on UI.
- **Testability**: The ViewModel can be easily tested without needing the UI.
- **Reactivity**: With SwiftUI's data-binding features, changes in the ViewModel automatically update the UI.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

How would you refactor a massive ViewController?

A **massive ViewController** often arises when the controller contains too much logic, making it hard to maintain and test. To refactor it, we can apply design patterns like **MVVM**, **Clean Architecture**, or **Coordinator Pattern**.

Step-by-Step Refactor:

1. **Identify Responsibilities**:
    - The ViewController should only be responsible for managing the view lifecycle and user interactions. All other logic (data fetching, business logic) should be moved out.
2. **Separate Concerns**:
    - Move **business logic** to a **ViewModel** (using MVVM).
    - Move **networking** code into a separate **service layer**.
    - Move **navigation logic** to a **Coordinator** (for complex navigation scenarios).
3. **Example**:
   Let's assume the original ViewController has responsibilities like handling networking, parsing JSON, and updating the UI. We'll refactor this by moving the networking code to a separate service and business logic to a ViewModel.

Original ViewController:

```
class UserProfileViewController: UIViewController {
    var nameLabel: UILabel!
    var profileImageView: UIImageView!

    var user: User? // Assume user model

    override func viewDidLoad() {
        super.viewDidLoad()
        loadUserProfile()
    }

    func loadUserProfile() {
        // Networking code here (bad practice)
        APIService.fetchUserProfile { [weak self] user in
            self?.user = user
            self?.updateUI()
        }
    }
```


Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
    func updateUI() {
        nameLabel.text = user?.name
        // Update other UI elements
    }
}
```

Refactor:

Move Networking to a Service:

```
class APIService {
    static func fetchUserProfile(completion: @escaping (User) -> Void)
{
        // Network call code
        completion(User(name: "John Doe"))
    }
}
```

Move Business Logic to ViewModel:

```
class UserProfileViewModel: ObservableObject {

    @Published var user: User?

    func loadUserProfile() {
        APIService.fetchUserProfile { [weak self] user in
            self?.user = user
        }
    }
}
```

Simplify ViewController:

```
class UserProfileViewController: UIViewController {
    var nameLabel: UILabel!
```

```
    var viewModel = UserProfileViewModel()

    override func viewDidLoad() {
        super.viewDidLoad()
        viewModel.loadUserProfile()
        bindViewModel()
    }

    func bindViewModel() {
        viewModel.$user.sink { [weak self] user in
            self?.nameLabel.text = user?.name
        }
    }
}
```

Benefits of Refactoring:

- **Single Responsibility Principle (SRP)**: The ViewController no longer handles networking and logic, making it easier to maintain.
- **Testability**: The ViewModel and service can now be tested independently of the ViewController.
- **Separation of Concerns**: The UI code is now clean and focused solely on view management, while the data and business logic are in their own layers.

Summary:

- **MVVM** divides the app into three layers: Model, View, and ViewModel, making it easier to manage data, UI, and logic separately.
- **Refactoring a massive ViewController** involves moving responsibilities out of the controller into separate components like ViewModels, Services, and Coordinators to improve maintainability and testability.

Concurrency in Swift – Explained with Examples

What's the difference between GCD and OperationQueue?

Feature	GCD (Grand Central Dispatch)	OperationQueue

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

API Style	C-based, lightweight	OOP-based, more flexible
Dependencies	❌ No direct support	Supports dependencies
Priorities	With QoS (Quality of Service)	Supports priorities
Cancellation	❌ Not cancelable	Can cancel operations
Reusability	Not reusable	Can subclass `Operation`
Use Case	Quick async tasks	Complex task chaining, cancellation

Example – GCD:

```
DispatchQueue.global(qos: .background).async {
    let result = performHeavyTask()
    DispatchQueue.main.async {
        self.label.text = result
    }
}
```

Example – OperationQueue:

```
let queue = OperationQueue()
let operation = BlockOperation {
    let result = performHeavyTask()
    OperationQueue.main.addOperation {
        self.label.text = result
    }
}
queue.addOperation(operation)
```

How do you manage background tasks in SwiftUI?

In SwiftUI, you generally use `Task` or `.task` modifiers with `async/await` to handle background operations.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Example – API Call in Background with UI Update:

```
struct ContentView: View {
    @State private var data = ""

    var body: some View {
        Text(data)
            .task {
                await fetchData()
            }
    }

    func fetchData() async {
        if let result = await NetworkService.getData() {
            data = result // UI update on main thread
        }
    }
}
```

Other tools:

- `Task(priority:) { ... }` for background priorities
- `URLSession.shared.data(for:)` with async/await
- `.background` for view modifiers if needed

How would you handle an API call that needs to return on the main thread?

You **always update the UI on the main thread**. With `async/await`, Swift ensures this is safe if you're using `@MainActor`.

Best Practice using `@MainActor`:

```
@MainActor
class MyViewModel: ObservableObject {
    @Published var data: String = ""
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
    func fetch() async {
        let result = await APIService.fetchData()
        data = result // Automatically runs on main thread
    }
}
```

Without MainActor (Manual Dispatch):

```
func fetchData() {
    DispatchQueue.global().async {
        let result = NetworkService.load()
        DispatchQueue.main.async {
            self.data = result // Ensure main thread UI update
        }
    }
}
```

How do you make a network call in Swift?

In modern Swift (especially from Swift 5.5 onwards), the preferred way is using `URLSession` with `async/await`. Before that, we used closures or third-party libraries like Alamofire.

Example using `async/await` (Swift 5.5+):

```
struct Post: Codable {
    let id: Int
    let title: String
}

func fetchPosts() async throws -> [Post] {
    let url = URL(string:
"https://jsonplaceholder.typicode.com/posts")!
    let (data, _) = try await URLSession.shared.data(from: url)
    return try JSONDecoder().decode([Post].self, from: data)
}
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Using `URLSession` with completion handler (older way):

```
func fetchPosts(completion: @escaping ([Post]?) -> Void) {
    let url = URL(string:
"https://jsonplaceholder.typicode.com/posts")!
    URLSession.shared.dataTask(with: url) { data, response, error in
        guard let data = data else {
            completion(nil)
            return
        }
        let posts = try? JSONDecoder().decode([Post].self, from: data)
        completion(posts)
    }.resume()
}
```

How do you handle decoding errors with `Codable`?

Swift's `Codable` decoding can fail for many reasons (missing keys, type mismatches, invalid formats). To handle this, wrap decoding in `do-catch` and optionally inspect the error.

Example with Error Handling:

```
do {
    let decoded = try JSONDecoder().decode([Post].self, from: data)
    return decoded
} catch let DecodingError.keyNotFound(key, context) {
    print("Missing key: \(key), context: \(context)")
} catch let DecodingError.typeMismatch(type, context) {
    print("Type mismatch: \(type), context: \(context)")
} catch {
    print("Decoding failed: \(error.localizedDescription)")
}
```

Tip: Use `debugDescription` or `localizedDescription` to log detailed errors.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Do you use third-party libraries (like Alamofire)? Why or why not?

Yes, when appropriate.

Why use Alamofire:

- Cleaner syntax for chaining requests
- Built-in support for multipart uploads, background sessions
- Handles SSL pinning, retry policies, and network reachability
- Better abstraction for request/response validation

Why avoid it sometimes:

- Adds external dependency and app size
- Native `URLSession` + `async/await` now covers most needs
- Fine-grained control over URLSession behavior

My Practice:

- **Small/medium apps** → Native `URLSession` with `async/await`
- **Large apps or teams** → Alamofire or combine it with **Moya** for better network abstraction layer

---

What persistence methods have you used?

In iOS, there are several common persistence methods, each suited to different use cases:

UserDefaults:

- Used for lightweight data storage (simple key-value pairs).
- Best for small settings or configurations (e.g., preferences, flags).

Example:

```
// Save data
UserDefaults.standard.set("dark", forKey: "theme")

// Retrieve data
let theme = UserDefaults.standard.string(forKey: "theme")
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

CoreData:

- A powerful framework for managing complex data models with relationships.
- Suitable for larger, structured data with many objects (e.g., complex models with entities and attributes).
- Offers querying, filtering, and sorting capabilities.

Example:

```
// Create and save an entity
let context = persistentContainer.viewContext
let newEntity = Entity(context: context)
newEntity.attribute = "value"

do {
    try context.save()
} catch {
    print("Failed to save context: \(error)")
}

// Fetch entities
let request: NSFetchRequest<Entity> = Entity.fetchRequest()
do {
    let entities = try context.fetch(request)
} catch {
    print("Failed to fetch entities: \(error)")
}
```

Realm:

- A fast, simple alternative to CoreData with an easier setup.
- Used for mobile apps needing fast local databases and cross-platform compatibility.
- Works with models like `Object` subclasses and supports both relational and flat data models.

Example:

```
import RealmSwift

class Cocktail: Object {
```

```
    @objc dynamic var name = ""
    @objc dynamic var isFavorite = false
}

let realm = try! Realm()

// Save a favorite cocktail
let cocktail = Cocktail()
cocktail.name = "Margarita"
cocktail.isFavorite = true

try! realm.write {
    realm.add(cocktail)
}

// Fetch all favorite cocktails
let favoriteCocktails =
realm.objects(Cocktail.self).filter("isFavorite == true")
```

SQLite:

- A relational database that can be used for large and structured data.
- Often used when you need direct SQL queries or prefer not to use higher-level libraries like CoreData or Realm.

How would you model and store favorite cocktails in your app?

Let's assume you're building an app to store favorite cocktails with attributes such as `name`, `ingredients`, and `isFavorite`. Here's how we can model and persist this data using **CoreData** and **Realm**:

CoreData Example:

Define the CoreData model: Create a `Cocktail` entity with the following attributes:

- `name`: String

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

- `ingredients`: String (or use a relationship if you want to store ingredients as a separate entity)
- `isFavorite`: Boolean

Create a `Cocktail` object in CoreData:

```
import CoreData

func addCocktail(name: String, ingredients: String, isFavorite: Bool)
{
    let context = persistentContainer.viewContext
    let newCocktail = Cocktail(context: context)
    newCocktail.name = name
    newCocktail.ingredients = ingredients
    newCocktail.isFavorite = isFavorite

    do {
        try context.save()
    } catch {
        print("Failed to save cocktail: \(error)")
    }
}
```

Fetch favorite cocktails:

```
 func fetchFavoriteCocktails() -> [Cocktail] {

    let context = persistentContainer.viewContext
    let fetchRequest: NSFetchRequest<Cocktail> =
Cocktail.fetchRequest()
    fetchRequest.predicate = NSPredicate(format: "isFavorite == true")

    do {
        let favoriteCocktails = try context.fetch(fetchRequest)
        return favoriteCocktails
    } catch {
        print("Failed to fetch favorite cocktails: \(error)")
        return []
```

```
    }
}
```

Realm Example:

Define the `Cocktail` model:

```
import RealmSwift

class Cocktail: Object {
    @objc dynamic var name = ""
    @objc dynamic var ingredients = ""
    @objc dynamic var isFavorite = false
}
```

Save a cocktail to Realm:

```
 func addCocktailToRealm(name: String, ingredients: String,
 isFavorite: Bool) {

    let cocktail = Cocktail()
    cocktail.name = name
    cocktail.ingredients = ingredients
    cocktail.isFavorite = isFavorite

    let realm = try! Realm()

    try! realm.write {
        realm.add(cocktail)
    }
}
```

Fetch favorite cocktails:

```
 func fetchFavoriteCocktailsFromRealm() -> Results<Cocktail> {

    let realm = try! Realm()
    return realm.objects(Cocktail.self).filter("isFavorite == true")
```

```
}
```

How do you write unit tests in Swift?

Unit testing in Swift is done using the **XCTest** framework, which is integrated into Xcode. XCTest allows you to write tests that validate the correctness of your code, helping ensure that each component works as expected.

Basic Unit Test Example:

To write a unit test, you'll usually create a new test target in Xcode. Here's a simple example of a test case for a function that adds two numbers.

Testable Code Example:

```
class Calculator {
    func add(_ a: Int, _ b: Int) -> Int {
        return a + b
    }
}
```

Writing Unit Test: In your test file, you would write a test case for the `add` function.

```
import XCTest

@testable import YourApp

class CalculatorTests: XCTestCase {

    var calculator: Calculator!

    override func setUp() {
        super.setUp()
        calculator = Calculator()
    }

    override func tearDown() {
        calculator = nil
        super.tearDown()
```

```
    }

    func testAdd() {
        let result = calculator.add(2, 3)
        XCTAssertEqual(result, 5, "Add function should return the sum
of two numbers")
    }
}
```

- **`setUp()`**: Used to set up any resources before each test method is called.
- **`tearDown()`**: Used to clean up resources after each test.
- **`XCTAssertEqual`**: A built-in assertion that checks if the two values are equal.

Running Tests:

In Xcode, you can run tests by pressing **Command + U**, or running tests from the Test Navigator.

What tools do you use for UI testing?

For **UI Testing** in Swift, we primarily use the **XCUITest** framework, which is part of the XCTest framework. It helps simulate user interactions and validate that the UI behaves as expected.

UI Test Example:

Here's an example of a simple UI test where we check if a button's tap changes a label's text.

1. **UI Test Setup:** In Xcode, create a new **UI Test** target for your app, which generates a test class for UI testing.

Test Example:

```
import XCTest

class MyAppUITests: XCTestCase {

    func testButtonTapChangesLabelText() {
        let app = XCUIApplication()
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
        app.launch()

        let button = app.buttons["MyButton"]
        let label = app.staticTexts["MyLabel"]

        // Check initial label text
        XCTAssertEqual(label.label, "Initial Text")

        // Tap the button
        button.tap()

        // Verify if the label text changes after button tap
        XCTAssertEqual(label.label, "Text After Tap")
    }
}
```

- **`XCUIApplication`**: Represents the app being tested.
- **`app.buttons["MyButton"]`**: Access the button by its accessibility identifier.
- **`button.tap()`**: Simulate a tap action on the button.
- **`XCTAssertEqual`**: Verify the label's text after the button is tapped.

Accessibility Identifiers:

Always set accessibility identifiers for UI elements to make them easier to identify during tests.

```
button.accessibilityIdentifier = "MyButton"
label.accessibilityIdentifier = "MyLabel"
```

How would you test an async network call?

Testing asynchronous network calls requires simulating network responses and waiting for the call to finish using expectations.

Example of Testing an Async Network Call:

Let's assume we have an API service that fetches data from a server, and we want to write a test to ensure that it works correctly.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Network Call Example (Using `async/await`):

```
class NetworkService {
    func fetchData() async throws -> String {
        let url = URL(string: "https://api.example.com/data")!
        let (data, _) = try await URLSession.shared.data(from: url)
        return String(data: data, encoding: .utf8) ?? ""
    }
}
```

Unit Test for Async Network Call:

```
import XCTest
@testable import YourApp

class NetworkServiceTests: XCTestCase {

    var networkService: NetworkService!

    override func setUp() {
        super.setUp()
        networkService = NetworkService()
    }

    override func tearDown() {
        networkService = nil
        super.tearDown()
    }

    func testFetchData() async {
        // Create an expectation
        let expectation = XCTestExpectation(description: "Async
network call should complete successfully")

        do {
            let result = try await networkService.fetchData()
            XCTAssertEqual(result, "Expected Result")
            expectation.fulfill() // Fulfill the expectation once the
async call finishes
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
        } catch {
            XCTFail("Network call failed with error: \(error)")
        }

        // Wait for expectations with a timeout (important for async
tests)
        await wait(for: [expectation], timeout: 5.0)
    }
}
```

- **`XCTestExpectation`**: Creates an expectation for asynchronous code.
- **`expectation.fulfill()`**: Marks the expectation as completed.
- **`wait(for:timeout:)`**: Waits for the expectation to be fulfilled within a given timeout.

Mocking Network Calls:

For unit tests, you can use libraries like **OHHTTPStubs** to mock network responses and avoid actual API calls during tests.

Problem Solving / Code Review

example of a piece of code that might be provided in a coding interview or task, followed by the task to refactor it or spot issues:

Example Code (Unoptimized and Problematic):

```
func getEvenNumbers(numbers: [Int]) -> [Int] {
    var result = [Int]()
    for number in numbers {
        if number % 2 == 0 {
            result.append(number)
        }
    }
    return result
}

func getSumOfEvenNumbers(numbers: [Int]) -> Int {
    var sum = 0
    for number in numbers {
        if number % 2 == 0 {
            sum += number
        }
    }
    return sum
}
```

Task: Refactor the code and identify any issues

Potential Issues & Areas for Improvement:

1. **Code Duplication**: Both functions (`getEvenNumbers` and `getSumOfEvenNumbers`) are iterating over the same `numbers` array and checking if the number is even. This can be optimized by avoiding the duplication of logic.
2. **Inefficiency**: The current code processes the list twice—once to collect even numbers and once to calculate the sum of even numbers. This could be done in a single iteration.
3. **Readability**: The code could be more readable by using higher-order functions like `filter` and `reduce` in Swift.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Refactored Code:

```
func getEvenNumbers(numbers: [Int]) -> [Int] {
    return numbers.filter { $0 % 2 == 0 }
}

func getSumOfEvenNumbers(numbers: [Int]) -> Int {
    return numbers.filter { $0 % 2 == 0 }.reduce(0, +)
}
```

Explanation of Refactor:

1. **Using `filter`**: The `filter` method is used to create an array of even numbers. It's a more declarative and concise approach than manually iterating and appending to a result array.
2. **Using `reduce`**: The `reduce` method is used to calculate the sum of even numbers in one pass. It accumulates the sum, starting with `0` and adding up all even numbers.

Final Thoughts:

- The refactored code is more concise and efficient.
- It eliminates redundant loops by combining operations into one pass for filtering and summing.
- The use of `filter` and `reduce` improves readability and makes the code more Swift-idiomatic.

Duplicated Logic in Loop

Code:

```
func calculateSquareOfEvenNumbers(numbers: [Int]) -> [Int] {
    var result = [Int]()
    for number in numbers {
        if number % 2 == 0 {
            result.append(number * number)
        }
    }
    return result
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
}

func calculateSquareOfOddNumbers(numbers: [Int]) -> [Int] {
    var result = [Int]()
    for number in numbers {
        if number % 2 != 0 {
            result.append(number * number)
        }
    }
    return result
}
```

Issue: Duplicated loop logic. **Refactor:** Combine the logic into a single function and use conditional checks.

Inefficient String Concatenation

Code:

```
func concatenateStrings(strings: [String]) -> String {
    var result = ""
    for string in strings {
        result += string
    }
    return result
}
```

Issue: Inefficient string concatenation in a loop. **Refactor:** Use `join` instead of `+=` for better performance.

Unnecessary Nested Loops

Code:

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
func checkForDuplicates(numbers: [Int]) -> Bool {
    for i in 0..<numbers.count {
        for j in i+1..<numbers.count {
            if numbers[i] == numbers[j] {
                return true
            }
        }
    }
    return false
}
```

Issue: Nested loops can be avoided for checking duplicates. **Refactor:** Use a `Set` to eliminate duplicates and improve performance.

Unoptimized Array Search

Code:

```
func findMaxNumber(numbers: [Int]) -> Int? {
    var maxNumber: Int? = nil
    for number in numbers {
        if maxNumber == nil || number > maxNumber! {
            maxNumber = number
        }
    }
    return maxNumber
}
```

Issue: Iterating manually for maximum value. **Refactor:** Use the built-in `max()` method.

Redundant Conditional Checks

Code:

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
func isAdult(age: Int) -> Bool {
    if age >= 18 {
        return true
    } else {
        return false
    }
}
```

Issue: The code is redundant. **Refactor:** Return the condition directly.

Inconsistent Return Type

Code:

```
func getDescription(age: Int) -> String {
    if age >= 18 {
        return "Adult"
    } else {
        return 18
    }
}
```

Issue: Returning different types (`String` and `Int`). **Refactor:** Ensure the return type is consistent.

Inefficient Dictionary Search

Code:

```
func findName(forId id: Int, in dictionary: [Int: String]) -> String?
{
    for key in dictionary.keys {
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
        if key == id {
            return dictionary[key]
        }
    }
    return nil
}
```

Issue: Inefficient dictionary search. **Refactor:** Directly access the dictionary using the key.

Incorrect Optional Unwrapping

Code:

```
func divideNumbers(_ a: Int, _ b: Int) -> Int? {
    if b == 0 {
        return nil
    }
    return a / b
}
```

Issue: Force unwrapping the optional result. **Refactor:** Use optional binding to handle safe unwrapping.

Overcomplicated Sorting

Code:

```
func sortNumbers(numbers: [Int]) -> [Int] {
    var sortedNumbers = numbers
    for i in 0..<numbers.count {
        for j in 0..<numbers.count-i-1 {
            if sortedNumbers[j] > sortedNumbers[j+1] {
                let temp = sortedNumbers[j]
                sortedNumbers[j] = sortedNumbers[j+1]
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
                sortedNumbers[j+1] = temp
            }
        }
    }
    return sortedNumbers
}
```

Issue: Using bubble sort when `sorted()` is available. **Refactor:** Use `sorted()` for simplicity and efficiency.

Unnecessary Array Index Access

Code:

```
func getFirstElement(numbers: [Int]) -> Int? {
    if numbers.count > 0 {
        return numbers[0]
    } else {
        return nil
    }
}
```

Issue: Unnecessary check for array length. **Refactor:** Access the array directly and use optional binding.

Manual JSON Parsing

Code:

```
func parseUserData(data: Data) -> User? {
    let json = try? JSONSerialization.jsonObject(with: data, options:
[]) as? [String: Any]
    if let jsonDict = json {
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
        if let name = jsonDict["name"] as? String, let age =
jsonDict["age"] as? Int {
            return User(name: name, age: age)
        }
    }
    return nil
}
```

Issue: Manual parsing can be error-prone. **Refactor:** Use `Codable` and `JSONDecoder` for safer, cleaner parsing.

Inefficient Filtering

Code:

```
func filterEvenNumbers(numbers: [Int]) -> [Int] {
    var result = [Int]()
    for number in numbers {
        if number % 2 == 0 {
            result.append(number)
        }
    }
    return result
}
```

Issue: Can use the `filter` method for better performance. **Refactor:** Use `filter` instead of manual iteration.

Repetitive String Matching

Code:

```
func hasSubstring(text: String, substring: String) -> Bool {
    if text.contains(substring) {
```

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

```
        return true
    } else {
        return false
    }
}
```

Issue: Redundant conditional check. **Refactor:** Return the result of `contains` directly.

Unnecessary Type Casting

Code:

```
func addNumbers(a: Any, b: Any) -> Int? {
    if let num1 = a as? Int, let num2 = b as? Int {
        return num1 + num2
    }
    return nil
}
```

Issue: Type casting may be avoided with correct function parameters. **Refactor:** Change the function parameters to `Int`directly to avoid unnecessary type casting.

Too Many Nested Ternary Operators

Code:

```
func checkAge(age: Int) -> String {
    return age < 18 ? "Minor" : age < 21 ? "Young Adult" : "Adult"
}
```

Issue: Too many nested ternary operators can reduce readability. **Refactor:** Use if-else for better clarity.

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)

Created By : Ruchit B Shah
(Senior Principal Software Engineer - Mobile Developer : + 91 9228877722)